

CHAPTER 09

09. 인증 — 비밀번호 해시와 JWT 토큰

Node.js · Express · NestJS 강의 · 2026

지금까지 만든 게시판에는 빠진 게 하나 있다. "누가 썼는가" 다. 아무나 `POST /posts` 를 보내면 글이 만들어지고, 작성자도 클라이언트가 보낸 값을 그대로 믿었다. 실제 서비스라면 로그인한 사람만 글을 쓰게 막아야 하고, 작성자도 서버가 직접 확인한 신원으로 채워야 한다.

로그인은 어떻게 동작하나. 큰 흐름은 두 단계다. 먼저 **회원가입** — 사용자가 이메일과 비밀번호를 보내면 서버가 저장한다. 이때 비밀번호를 그대로 저장하면 안 된다. DB가 유출되면 모든 사람의 비밀번호가 통째로 새기 때문이다. 그래서 **해시**로 바꿔 저장한다. 다음은 **로그인** — 사용자가 다시 이메일과 비밀번호를 보내면, 서버는 저장된 해시와 비교해 맞으면 **토큰 (JWT)** 을 발급한다. 이후 클라이언트는 글을 쓸 때마다 이 토큰을 헤더에 실어 보내고, 서버는 토큰을 검증해 "아, 이 사람은 로그인한 김철수구나"를 알아낸다.

이 장은 Express 파트의 마지막이다. 08장에서 컨트롤러·서비스·모델로 계층을 나눴다면, 이번에는 거기에 **인증**을 더한다.

이 자료는 **개념 예습용**이다. 실제 수업에서 쓰는 파일 이름과 폴더 구성, 다루는 범위는 강사에 따라 다를 수 있다. 그래서 여기서는 파일이 아니라 **무엇을 배우는가**에 초점을 둔다.

이 장에서 배우는 것

- 비밀번호를 **해시**로 저장하는 이유와 `bcrypt` 의 `hash` · `compare`
- **토큰 기반 인증**이 무엇이고, JWT의 `sign` (발급) · `verify` (검증)이 어떻게 동작하는지
- `Authorization: Bearer <token>` 헤더에서 토큰을 꺼내 검증하는 **인증 미들웨어**
- 라우트 하나에만 미들웨어를 끼워 **목록은 공개·글쓰기는 인증 필요**로 나누는 법
- 작성자를 클라이언트가 아니라 **검증된 토큰**에서 채우는 이유

내용은 세 주제로 나뉜다. 앞의 두 주제는 서버가 아니라 **콘솔에 결과만 찍는** 학습용 코드로 따로 떼어 눈으로 확인하고, 마지막 주제에서 둘을 합쳐 진짜 인증 API를 만든다.

주제	핵심
비밀번호 해시	왜 평문 저장 금지인가 — <code>bcrypt</code> 해시 / 비교
JWT 기초	JWT 발급(<code>sign</code>) / 검증(<code>verify</code>) / 위조 거부
보호 라우트	가입 → 로그인 → 토큰 → 로그인한 사람만 글쓰기

핵심 개념 — 해시 그리고 토큰

인증을 떠받치는 두 기둥이 있다. 비밀번호를 안전하게 보관하는 **해시**, 그리고 로그인 상태를 증명하는 **토큰**이다. 코드를 보기 전에 이 둘부터 잡고 간다.

해시는 암호화가 아니다

둘 다 원문을 알아보기 어려운 형태로 바꾸지만, 결정적인 차이가 있다. **암호화는 되돌릴 수 있고, 해시는 되돌릴 수 없다**. 암호화는 열쇠(키)가 있으면 원문을 복원하는 게 목적이다. 반면 해시는 한 방향이다. 같은 입력은 항상 같은 결과를 내지만, 결과만 보고 원래 입력을 거꾸로 알아낼 방법이 없다.

비밀번호 저장에 해시를 쓰는 이유가 여기 있다. 서버조차 사용자의 원래 비밀번호를 알 필요가 없다. 로그인할 때 입력값을 똑같이 해시해서, 저장된 해시와 **같은지만 비교하면** 된다. DB가 통째로 유출돼도 공격자가 손에 쥐는 건 해시뿐이고, 거기서 원래 비밀번호를 복원하지 못한다.

솔트(salt)란 — 같은 비밀번호는 같은 해시가 되니, 미리 계산해 둔 해시 사전(레인보우 테이블)으로 뚫릴 수 있다. 그래서 bcrypt는 매번 임의의 값(솔트)을 섞어 해시한다. 덕분에 같은 비밀번호라도 가입할 때마다 해시가 달라진다. 솔트는 해시 결과 안에 함께 들어가므로 따로 보관하지 않아도 된다.

토큰 기반 인증이란

로그인에 성공한 다음이 문제다. HTTP는 요청 하나하나가 서로를 기억하지 못하는 **무상태(stateless)** 구조다. 방금 로그인한 사람이 다음 요청에서 또 글을 쓰려 해도, 서버 입장에선 둘이 같은 사람인지 알 길이 없다. 매 요청마다 "나 아까 로그인한 그 사람"임을 증명할 무언가가 필요하다.

그 증명서가 **토큰**이다. 로그인이 성공하면 서버는 "이 토큰을 가진 사람은 id 1번 김철수"라는 내용을 담은 토큰을 발급한다. 클라이언트는 이걸 보관해 두었다가, 이후 요청마다 헤더에 실어 보낸다. 서버는 토큰만 보면 누구인지 바로 안다. 세션처럼 서버가 로그인 상태를 따로 기억할 필요가 없다.

여기서 쓰는 토큰 형식이 **JWT(JSON Web Token)** 다. 생김새는 점 두 개로 나뉜 세 토막이다.

```
헤더.페이로드.서명
eyJhbGciOiAiOyJpZCI6IiwiaWF0IjoxNjU4MjU0MDAwfQ.eyJpZCI6IiwiaWF0IjoxNjU4MjU0MDAwfQ.eyJpZCI6IiwiaWF0IjoxNjU4MjU0MDAwfQ.
```

- **페이로드(payload)** — 토큰에 담은 내용. `{ id: 1, name: "김철수" }` 같은 사용자 정보가 들어간다. **누구나 디코딩해 볼 수 있다**. 암호화가 아니므로 비밀번호처럼 민감한 값은 절대 담지 않는다.
- **서명(signature)** — 서버만 아는 **비밀키(secret)** 로 헤더와 페이로드를 봉인한 부분이다. 누가 페이로드를 살짝 고쳐도 서명이 맞지 않아 검증에서 걸린다. 토큰의 안전은 "내용을 숨기는 것"이 아니라 "위조를 막는 것"에 있다.

페이로드는 열려 있고, 서명이 그 내용의 진위를 보장한다. 비밀키를 모르면 유효한 서명을 만들 수 없으니, 남이 토큰을 위조하지 못한다.

콜아웃을 활용하라 — 이 장의 코드 주석에는 "왜 이렇게 했는가"가 빼곡히 달려 있다. 솔트 라운드를 왜 10으로 뒀는지, 로그인 실패에 왜 어느 쪽이 틀렸는지 안 알려주는지, 작성자를 왜 토큰에서 채우는지 — 보안 코드는 한 줄의 선택에 이유가 담긴다. 주석과 아래 해설을 함께 읽으면 "관행"이 "이유"로 바뀐다.

준비

예제 폴더에서 `npm install` 로 의존성을 설치한다. 이 장에 새로 필요한 건 셋이다.

- `express` — 04장부터 써 온 웹 프레임워크. 인증 API 서버를 띄운다.
- `bcryptjs` — 비밀번호를 해시하고 비교한다. 순수 자바스크립트 구현이라 별도 빌드 도구 없이 설치된다.
- `jsonwebtoken` — JWT를 발급(`sign`)하고 검증(`verify`)한다.

해시와 JWT 기초는 서버가 아니라 콘솔 코드다. 실행하면 결과를 찍고 바로 끝난다. 인증 API는 서버이므로 띄워 두고 `curl` 로 요청을 보낸다. 종료는 `Ctrl+C` 다.

비밀번호 해시 (bcrypt)

평문 저장이 왜 금지인지, 그리고 해시가 어떻게 그 문제를 푸는지를 코드로 확인한다. 서버 없이 콘솔에만 결과를 찍는다.

```
const bcrypt = require("bcryptjs");

async function main() {
  const password = "my-secret-1234";

  // 저장할 때: 해시한다. (10 = salt 라운드, 클수록 느리고 안전)
  const hash = await bcrypt.hash(password, 10);
  console.log("평문:", password);
  console.log("해시:", hash);
  console.log("→ DB엔 평문이 아니라 이 해시를 저장한다.");

  // 로그인할 때: 입력값이 저장된 해시와 맞는지 비교한다(원문 복원이 아니라 비교).
  console.log("올바른 비밀번호?", await bcrypt.compare("my-secret-1234", hash)); // true
  console.log("틀린 비밀번호?", await bcrypt.compare("wrong-pw", hash)); // false
}

main();
```

핵심은 `bcrypt` 의 두 함수다.

- `bcrypt.hash(password, 10)` — 평문을 해시로 바꾼다. 두 번째 인자 `10` 은 솔트 라운드다. 해시를 몇 번 반복해 돌릴지를 정하는데, 숫자가 클수록 계산이 느려진다. 느린 게 단점 같지만 보안에선 장점이다. 공격자가 비밀번호를 무차별로 대입할 때도 한 번 시도에 그만큼 오래 걸려서다. `10` 은 안전과 속도의 균형이 무난한 기본값이다.
- `bcrypt.compare(입력값, 해시)` — 입력한 평문을 같은 방식으로 해시해서 저장된 해시와 맞는지 비교한다. 맞으면 `true` , 틀리면 `false` . 해시를 원문으로 되돌리는 게 아니라, 새로 해시해 같은지만 따진다는 점이 중요하다.

두 함수 모두 `await` 를 붙인 점에 주목하자. `bcrypt`는 일부러 느리게 도는 계산이라 비동기로 동작한다. 그래서 결과를 기다리려면 `await` 가 필요하고, 그 `await` 를 쓰려고 코드를 `async function main()` 으로 감쌌다.

실행하면 이런 결과가 나온다.

```

평문: my-secret-1234
해시: $2a$10$N9qo8uLOickgx2ZMRZoMy.
→ DB엔 평문이 아니라 이 해시를 저장한다.
올바른 비밀번호? true
틀린 비밀번호? false

```

해시 문자열을 보면 `$2a$10$` 으로 시작한다. `bcrypt` 방식이라는 표시(`2a`)와 솔트 라운드(`10`)가 해시 안에 함께 박혀 있다. 같은 비밀번호로 다시 실행하면 해시 값이 또 달라진다. 솔트가 매번 바뀌기 때문이다. 그래도 `compare` 는 정확히 통과한다. 솔트가 해시 안에 들어 있어서다.

짚고 가기 — 여기서 본 `hash` 와 `compare` 가 곧 회원가입과 로그인인 두 동작이다. 가입할 때 `hash` 로 저장하고, 로그인할 때 `compare` 로 확인한다. 뒤의 인증 API에서 그대로 다시 만난다.

JWT 발급과 검증

토큰을 만들고(`sign`), 확인하고(`verify`), 위조된 토큰이 어떻게 걸러지는지를 본다. 이 주제도 콘솔 코드다.

```

const jwt = require("jsonwebtoken");

// 참고: bcrypt 해시는 계산이 무거워 비동기라 await로 기다렸지만, jwt.sign/verify는 동기 함수라 await가 필요 없다.
const SECRET = "my-secret-key"; // 실무에선 .env로 분리해 숨긴다

// 발급(sign) - 로그인 성공 시 사용자 정보를 담아 토큰을 만든다.
const token = jwt.sign({ id: 1, name: "김철수" }, SECRET, { expiresIn: "1h" });
console.log("토큰:", token);

// 검증(verify) - 요청이 올 때 토큰이 진짜인지 확인하고 내용을 꺼낸다.
const payload = jwt.verify(token, SECRET);
console.log("검증 통과, 내용:", payload); // { id: 1, name: '김철수', iat, exp }

```

- **SECRET** — 토큰에 서명할 비밀키다. 이 키를 아는 쪽(=서버)만 유효한 토큰을 만들거나 검증한다. 지금은 코드에 박아 뒀지만 실무에선 절대 그러면 안 된다. 이 점은 뒤의 '흔한 실수'에서 다시 다룬다.
- **jwt.sign(페이로드, SECRET, 옵션)** — 토큰을 발급한다. 첫 인자 `{ id: 1, name: "김철수" }` 가 페이로드, 즉 토큰에 담을 내용이다. 세 번째 옵션 `{ expiresIn: "1h" }` 는 유효 기간을 1시간으로 정한다. 토큰이 영원히 유효하면 한 번 새어 나갔을 때 막을 방법이 없으니, 만료 시간을 뒤서 위험을 줄인다.
- **jwt.verify(token, SECRET)** — 토큰이 진짜인지 확인하고, 통과하면 페이로드를 꺼내 돌려준다. 결과를 보면 우리가 넣은 `id · name` 말고 `iat` (발급 시각)와 `exp` (만료 시각)가 자동으로 붙어 있다. `expiresIn` 옵션이 `exp` 를 채운 것이고, `verify` 는 현재 시각이 `exp` 를 넘었는지도 함께 확인한다.

`sign` 과 `verify` 어디에도 `await` 가 없다. `bcrypt`와 달리 JWT 계산은 가벼워서 동기로 동작한다. 앞의 해시와 헛갈리기 쉬운 지점이라 코드 주석에도 따로 짚어 뒀다.

이제 위조를 시도해 본다.

```
// 위조되거나 잘못된 토큰은 verify에서 에러가 난다.
try {
  jwt.verify(token + "x", SECRET);
} catch (err) {
  console.log("위조 토큰 거부:", err.message);
}
```

토큰 끝에 글자 하나("x")를 붙여 살짝 망가뜨렸다. 그러면 서명이 더는 맞지 않으므로 `verify` 는 값을 돌려주는 대신 **에러를 던진다**. 그래서 `try.catch` 로 감쌌다. 이 "위조면 에러가 난다"는 성질이 뒤에 만들 인증 미들웨어의 뼈대가 된다. `verify` 를 `try` 안에서 부르고, 에러가 나면 거부하는 식이다.

실행 결과는 이렇다.

```
토큰: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwibmFtZSI6.
검증 통과, 내용: { id: 1, name: '김철수', iat: 1717400000, exp: 1717403600 }
위조 토큰 거부: invalid signature
```

짚고 가기 — 토큰 문자열을 jwt.io 같은 디코더에 붙여 넣으면 페이로드가 그대로 읽힌다. 암호화가 아니라는 걸 직접 확인할 수 있다. 그래서 페이로드엔 공개돼도 괜찮은 값(id, 이름)만 담고, 비밀번호 같은 민감 정보는 넣지 않는다.

인증 게시판 API — 가입·로그인·보호 라우트

앞의 두 조각을 합칠 차례다. 해시로 비밀번호를 저장하고, 토큰으로 로그인 상태를 증명한다. 그리고 **목록은 누구나, 글쓰기는 로그인한 사람만으로** 권한을 나눈다.

데이터는 인메모리 배열로 둔다. 06~08장에서 DB를 이미 배웠으니, 여기선 인증 흐름 자체에 집중하려는 의도다. 실무에선 08장의 계층/DB에 그대로 더하면 된다.

```
const express = require("express");
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");

const app = express();
app.use(express.json());

const SECRET = "my-secret-key"; // 실무에선 .env로 분리

const users = []; // { id, name, email, password(해시) }
const posts = [];
let userId = 1;
let postId = 1;
```

위부분은 04~08장에서 본 뼈대 그대로다. `express.json()` 으로 본문을 파싱하고, 사용자와 게시물을 담을 배열, id를 매길 카운터를 둔다. 달라진 건 `bcrypt` 와 `jwt` 를 함께 불러온 것, 그리고 `users` 배열의 `password` 에 해시가 들어간다는 점이다.

회원가입 — 비밀번호를 해시해 저장

```
// 회원가입 - 비밀번호는 해시해서 저장한다.
app.post("/register", async (req, res) => {
  const { name, email, password } = req.body;
  if (!name || !email || !password) {
    return res.status(400).json({ message: "name, email, password는 필수입니다." });
  }
  if (users.find((u) => u.email === email)) {
    return res.status(409).json({ message: "이미 가입된 이메일입니다." });
  }
  const hash = await bcrypt.hash(password, 10);
  const user = { id: userId++, name, email, password: hash };
  users.push(user);
  // 비밀번호(해시)는 응답에서 제외한다.
  res.status(201).json({ id: user.id, name: user.name, email: user.email });
});
```

위에서부터 차례로 막는다.

- 필수 필드가 비었으면 **400 Bad Request** — 04장에서 본 입력 검증과 같다.
- 이미 가입된 이메일이면 **409 Conflict** — "현재 상태와 충돌한다"는 뜻의 상태코드다. 중복 가입을 막는 자리에 알맞다.
- `await bcrypt.hash(password, 10)` — 핵심 한 줄이다. 받은 평문을 그대로 저장하지 않고 해시로 바꾼다. 앞에서 본 그 `hash` 다. 핸들러를 `async` 로 선언한 이유도 이 `await` 때문이다.
- 저장하는 `user` 의 `password` 엔 평문이 아니라 `hash` 가 들어간다.

- 응답엔 `id · name · email` 만 골라 담는다. 비밀번호 해시는 응답에서 뺀다. 해시라도 굳이 밖으로 내보낼 이유가 없다. 08장에서 응답 가공을 서비스 계층에서 했듯, 여기선 핸들러가 직접 필요한 필드만 추렸다.

로그인 — 비밀번호 비교 후 토큰 발급

```
// 로그인 - 비밀번호를 비교한 뒤 토큰을 발급한다.
app.post("/login", async (req, res) => {
  const { email, password } = req.body;
  if (!email || !password) {
    return res.status(400).json({ message: "email, password는 필수입니다." });
  }
  const user = users.find((u) => u.email === email);
  // 사용자가 없거나 비밀번호가 틀리면 똑같이 401 (어느 쪽이 틀렸는지 알려주지 않는다)
  if (!user || !(await bcrypt.compare(password, user.password))) {
    return res.status(401).json({ message: "이메일 또는 비밀번호가 올바르지 않습니다." });
  }
  const token = jwt.sign({ id: user.id, name: user.name }, SECRET, { expiresIn: "1h" });
  res.json({ token });
});
```

가입의 거울상이다. 가입이 `hash` 로 저장했다면, 로그인은 `compare` 로 확인한다.

- 이메일로 사용자를 찾고, `await bcrypt.compare(password, user.password)` 로 입력한 비밀번호가 저장된 해시와 맞는지 본다. 앞에서 본 그 `compare` 다.
- 조건이 영리하다. 사용자가 없든(`!user`) 비밀번호가 틀리든(`!compare`) 똑같이 401로 응답하고, 메시지도 "이메일 또는 비밀번호"로 뭉뚱그린다. 일부러 그렇게 했다. "이메일은 맞는데 비밀번호가 틀렸다"고 알려주면, 공격자가 어떤 이메일이 가입돼 있는지 캐낼 수 있어서다. 둘을 구분하지 않는 게 보안 관행이다.
- 통과하면 `jwt.sign(.)` 으로 토큰을 발급해 `{ token }` 으로 돌려준다. 페이로드엔 `id` 와 `name` 만 담는다 — 앞의 JWT 기초에서 본 그대로다. 클라이언트는 이 토큰을 받아 두었다가 다음 요청부터 헤더에 실어 보낸다.

인증 미들웨어 — 헤더에서 토큰을 꺼내 검증

이제 "토큰을 가진 사람만 통과"를 담당하는 문지기다. 04~05장에서 본 라우트 미들웨어와 똑같은 `(req, res, next)` 함수다.


```
// 인증 미들웨어 - "Authorization: Bearer <token>"을 검사해 req.user를 채운다.
// (04/05에서 본 라우트 미들웨어와 같은 (req,res,next) 함수다.)
function auth(req, res, next) {
  const header = req.headers.authorization || "";
  const token = header.startsWith("Bearer ") ? header.slice(7) : null; // 7 = "Bearer " 길이(공백 포함)
  if (!token) {
    return res.status(401).json({ message: "토큰이 필요합니다." });
  }
  try {
    req.user = jwt.verify(token, SECRET); // 통과하면 { id, name }이 req.user에 담긴다
    next();
  } catch (err) {
    // 서명이 틀리거나 만료된(앞에서 본 1h가 지난) 토큰이면 verify가 에러를 던져 여기로 온다.
    return res.status(401).json({ message: "유효하지 않은 토큰입니다." });
  }
}
```

한 줄씩 본다.

- **req.headers.authorization** — 토큰은 `Authorization` 헤더에 담겨 온다. 약속된 형식은 `Bearer <토큰>` 이다. `Bearer` 는 "이 토큰을 가진 자(bearer)에게 권한을 준다"는 표준 표기다. 헤더가 아예 없을 수도 있으니 `|| ""` 로 빈 문자열을 기본값으로 뒤서, 다음 줄의 `startsWith` 가 안전하게 동작하게 한다.
- **header.slice(7)** — 형식이 `Bearer` 로 시작하면 그 뒤의 실제 토큰만 잘라낸다. `7` 은 `"Bearer "` 의 길이다 — 알파벳 6자에 공백 한 칸을 더한 값이다. 형식이 안 맞으면 `null` 로 둔다.
- 토큰이 없으면 **401 Unauthorized** — "인증이 안 됐다"는 신호다.
- **req.user = jwt.verify(token, SECRET)** — 앞에서 본 그 `verify` 다. 토큰이 진짜고 만료되지 않았으면 페이로드(`{ id, name }`)를 돌려주고, 그걸 **req.user** 에 담는다. 이 한 줄이 핵심이다. 이후 핸들러는 `req.user` 만 보면 "지금 요청한 사람이 누구인지"를 안다.
- **next()** — 검증을 통과한 요청만 다음 단계(라우트 핸들러)로 넘긴다. 04장에서 본 그 `next()` 다.
- **catch** — `verify` 는 서명이 틀리거나 만료된 토큰이면 에러를 던진다(앞의 JWT 기초에서 확인한 성질). 그 에러를 잡아 401로 거부한다. `next()` 를 부르지 않으니 요청은 여기서 멈추고 핸들러까지 가지 못한다.

미들웨어가 하는 일을 한 줄로 줄이면 이렇다. 토큰을 검사해 통과한 요청에만 **req.user** 를 채우고 핸들러로 넘기며, 그 외엔 401로 막는다.

라우트 — 목록은 공개, 글쓰기는 인증

```
// 목록은 누구나 볼 수 있다(공개).
app.get("/posts", (req, res) => {
  res.json(posts);
});

// 글쓰기는 로그인한 사람만 - 05의 validate처럼 auth를 라우트 앞에 끼운다(통과한 요청만 핸들러로).
// (04의 app.use는 모든 요청에 걸리는 전역 미들웨어, 이렇게 핸들러 인자로 넣으면 이 라우트에서만 동작한다.)
app.post("/posts", auth, (req, res) => {
  const { title, content } = req.body;
  if (!title || !content) {
    return res.status(400).json({ message: "title과 content는 필수입니다." });
  }
  // 작성자는 토큰에서 가져온다(클라이언트가 보낸 값이 아니라 검증된 정보).
  // 여기서 author를 토큰의 name(문자열)으로 둔다 - 08처럼 관계로 없을 땐 post.userId = req.user.id 로
  // 잇는다.
  const post = { id: postId++, title, content, author: req.user.name };
  posts.push(post);
  res.status(201).json(post);
});
```

두 라우트가 권한을 나누는 방식을 잘 봐야 한다.

- **GET /posts** — 미들웨어 없이 핸들러만 있다. 누구나 목록을 본다.
- **POST /posts** — 핸들러 앞에 **auth** 가 끼어 있다. `app.post("/posts", auth, (req, res) => .{.})` — 경
로와 핸들러 사이에 미들웨어를 넣으면, 그 라우트에 들어오는 요청만 미들웨어를 먼저 거친다. **auth** 를 통과하지 못
한 요청은 핸들러에 닿지도 못하고 401로 돌아간다.

여기서 04장의 `app.use`(미들웨어) 와의 차이가 또렷해진다. `app.use` 는 모든 요청에 걸리는 전역 미들웨어였다. 반면
핸들러 인자로 직접 끼우면 그 라우트에서만 동작한다. 인증처럼 어떤 길은 막고 어떤 길은 열어야 할 때 쓰는 방식이다.

마지막 한 줄이 이 장 전체의 결론이다.

```
const post = { id: postId++, title, content, author: req.user.name };
```

작성자(**author**)를 클라이언트가 보낸 본문이 아니라 **req.user.name** 에서 가져온다. 04장에선 **author** 를
`req.body` 에서 받아 그대로 믿었다. 그건 누구나 아무 이름으로 사칭할 수 있다는 뜻이다. 이제는 다르다. `req.user` 는
서버가 토큰을 검증해 직접 알아낸 신원이다. 클라이언트가 본문에 어떤 이름을 끼워 넣든 무시하고, 토큰이 증명한 사람을
작성자라 박는다. 인증을 붙이는 진짜 이유가 바로 이 한 줄에 있다.

```
app.listen(3000, () => console.log("http://localhost:3000 에서 실행 중"));
```

서버를 3000번 포트에서 띄운다. 04장 이후 줄곧 같은 마무리다.

직접 돌려 보기

서버를 띄우고 가입 → 로그인 → 글쓰기를 순서대로 따라가면 흐름이 한눈에 들어온다.

```
# 1) 회원가입 (비밀번호는 해시되어 저장)
curl -X POST localhost:3000/register \
  -H "Content-Type: application/json" \
  -d '{"name":"김철수","email":"kim@example.com","password":"1234"}' # 201

# 2) 로그인 → 토큰 받기
curl -X POST localhost:3000/login \
  -H "Content-Type: application/json" \
  -d '{"email":"kim@example.com","password":"1234"}' # { "token":
  .. " }

# 3) 토큰 없이 글쓰기 → 거부
curl -X POST localhost:3000/posts \
  -H "Content-Type: application/json" -d '{"title":"제목","content":"내용"}' # 401

# 4) 토큰 넣고 글쓰기 → 성공 (작성자는 토큰에서 채워짐)
curl -X POST localhost:3000/posts \
  -H "Authorization: Bearer <위에서 받은 token>" \
  -H "Content-Type: application/json" -d '{"title":"제목","content":"내용"}' # 201

# 목록은 누구나 (공개)
curl localhost:3000/posts
```

3번과 4번을 비교하면 인증이 한 일이 분명해진다. 같은 글쓰기 요청인데, 토큰이 없으면 401로 막히고 토큰을 넣으면 201로 통과한다. 그리고 4번으로 만든 글을 목록에서 보면 `author` 가 "김철수"로 채워져 있다. 본문엔 작성자를 넣지 않았는데도다. 토큰이 그 자리를 메웠다.

인증 흐름 한눈에

단계	라우트	하는 일	핵심 함수
회원가입	POST /register	비밀번호를 해시해 저장	<code>bcrypt.hash</code>
로그인	POST /login	비밀번호 비교 후 토큰 발급	<code>bcrypt.compare</code> → <code>jwt.sign</code>
인증 검사	(미들웨어 <code>auth</code>)	헤더의 토큰을 검증해 <code>req.user</code> 채움	<code>jwt.verify</code>
글쓰기	POST /posts	인증 통과한 사람만, 작성자는 토큰에서	<code>auth</code> + <code>req.user</code>
목록	GET /posts	누구나 (공개)	없음

상태코드	뜻	이 장에서 언제
201	생성됨	가입 성공·글쓰기 성공
400	잘못된 요청	필수 필드 누락
401	인증 안 됨	로그인 실패·토큰 없음·토큰 위조/만료
409	충돌	이미 가입된 이메일

흔한 실수

- 비밀번호를 평문으로 저장한다. → DB 한 번 유출에 전원의 비밀번호가 샌다. 저장 전 반드시 `bcrypt.hash` 를 거친다.
- 시크릿 키를 코드에 하드코딩한다. → 이 장은 학습용이라 `const SECRET = "my-secret-key"` 로 박아 뒀지만, 실무에선 절대 금지다. 코드가 깃에 올라가면 키가 통째로 공개되고, 키를 아는 사람은 누구나 토큰을 위조한다. `.env` 파일로 분리해 `process.env.JWT_SECRET` 으로 읽고, `.env` 는 `.gitignore` 에 넣는다.
- 페이로드에 비밀번호 같은 민감 정보를 담는다. → JWT 페이로드는 누구나 디코딩해 읽는다. 암호화가 아니다. `id · name` 처럼 공개돼도 괜찮은 값만 담는다.
- 로그인 실패에서 "이메일이 틀렸다"와 "비밀번호가 틀렸다"를 구분해 알려준다. → 가입된 이메일을 캐는 단서가 된다. 둘을 똑같이 401로 뭉뚱그린다.
- 작성자를 클라이언트 본문에서 받는다. → 사칭이 가능하다. 검증된 `req.user` 에서 채운다.
- Bearer** 접두사를 안 떼고 토큰을 검증한다. → `verify` 에 `"Bearer eyJ. "` 통째로 넘어가 실패한다. `slice(7)` 로 접두사를 떼야 한다.

직접 해보기

- 토큰을 만료시켜 보자. JWT 발급의 `expiresIn` 을 `"1s"` 로 바꾸고, 토큰을 만든 뒤 잠깐 기다렸다가 `verify` 하면 어떤 에러가 나는지 확인해 본다. 인증 미들웨어 `auth` 가 이걸 어떻게 401로 잡는지 다시 읽어 보자.
- 내 글만 보는 라우트를 더해 보자. `GET /my-posts` 에 `auth` 를 끼우고, `posts` 중 `author === req.user.name` 인 것만 응답한다.
- 인증 API에 글 삭제 라우트(`DELETE /posts/:id`)를 추가하되, 작성자 본인만 지울 수 있게 막아 보자. `auth` 로 신원을 확인하고, 글의 `author` 와 `req.user.name` 이 다르면 **403 Forbidden**(권한 없음)으로 돌려준다. 401(인증 안 됨)과 403(인증은 됐지만 권한 없음)의 차이도 함께 짚어 본다.

정리

- 비밀번호는 해시로 저장한다. 해시는 되돌릴 수 없고, 로그인은 `bcrypt.compare` 로 같은지만 비교한다.

- **JWT**는 로그인 상태를 증명하는 토큰이다. `jwt.sign` 으로 발급하고 `jwt.verify` 로 검증하며, 페이로드는 공개·서명이 위조를 막는다.
- 인증 미들웨어는 `Authorization: Bearer` 헤더에서 토큰을 꺼내 검증하고, 통과한 요청에 `req.user` 를 채워 핸들러로 넘긴다.
- 미들웨어를 라우트에만 끼우면 **공개 길과 인증 길을 나눌 수 있다**. 작성자는 클라이언트가 아니라 검증된 토큰에서 채운다.

여기까지가 Express 파트(01~09)다. 다음 파트(**NestJS**)에서는 지금까지 손으로 쌓은 같은 게시판을, 구조와 규칙이 미리 정해진 NestJS로 다시 짓는다.